

FULL STACK DEVELOPMENT

FASE 4 | AULA 05 -

CONEXÃO COM BANCO DE DADOS INTERNOS / FIRESTORE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS	13

EMANDA

O QUE VEM POR AÍ?

Nesta aula, aprenderemos como conectar aplicativos móveis a bancos de dados internos e ao Firebase Firestore. Abordaremos a importância dos bancos de dados, a diferença entre bancos de dados locais e remotos, como configurar e usar Firestore em um aplicativo móvel, e as melhores práticas para gerenciar dados de forma eficiente e segura.



HANDS ON

Configuraremos o Firestore em um projeto móvel, começando pela criação de um projeto no Firebase Console e ativação do Firestore com as credenciais adequadas. Adicionaremos o SDK do Firebase ao projeto, configurando `google-services.json` e inicializando o Firebase no código do aplicativo.

Em seguida, aprenderemos a realizar operações CRUD (Create, Read, Update, Delete) com Firestore. Implementaremos a criação de documentos, adicionando novos registros a coleções; leitura de documentos, recuperando dados de coleções; atualização de documentos, modificando campos existentes; e exclusão de documentos, removendo registros desnecessários. Além disso, discutiremos boas práticas, como organização lógica dos dados em coleções e subcoleções, definição de regras de segurança para controle de acesso, uso de consultas eficientes para melhorar o desempenho e aproveitamento do suporte offline do Firestore para garantir funcionamento adequado mesmo sem conexão com a internet.

SAIBA MAIS

O que são Bancos de dados?

Bancos de dados são sistemas organizados para armazenar, gerenciar e recuperar informações de forma eficiente. Eles são projetados para lidar com grandes volumes de dados, suportando operações complexas de leitura e escrita. A estrutura de um banco de dados pode variar de simples tabelas relacionais a complexos sistemas distribuídos, dependendo das necessidades da aplicação.

Importância dos bancos de dados em aplicativos móveis

A **persistência de dados** é fundamental em aplicativos móveis, permitindo que as informações sejam armazenadas de forma permanente e recuperadas posteriormente, mesmo após o fechamento do aplicativo. Isso garante que os dados do usuário, configurações e estado do aplicativo sejam mantidos entre sessões, proporcionando uma experiência de usuário contínua e ininterrupta.

Os bancos de dados também **melhoram a eficiência** do gerenciamento e recuperação de dados, especialmente quando lidam com grandes volumes de informações. Eles utilizam índices, cache e outras técnicas de otimização para garantir que as operações de leitura e escrita sejam realizadas de forma rápida e eficiente.

Por fim, a **confiabilidade** é outra vantagem crucial dos bancos de dados. Eles garantem que os dados sejam armazenados de forma segura e possam ser acessados quando necessário. Mecanismos de backup, recuperação de desastres e replicação de dados são frequentemente utilizados para assegurar a integridade e disponibilidade dos dados.

Tipos de banco de dados

- **Bancos de dados internos**
 - SQLite: é um banco de dados relacional leve embutido na maioria dos sistemas operacionais móveis. É ideal para armazenar dados estruturados localmente devido à sua simplicidade e eficiência. O SQLite é amplamente utilizado em aplicações móveis por ser robusto,

de fácil configuração e oferecer suporte transacional completo, garantindo a integridade dos dados.

- Realm: um banco de dados móvel que oferece desempenho superior ao SQLite e é mais fácil de usar, possui consultas intuitivas. Ele é projetado especificamente para dispositivos móveis, oferecendo uma API simples e uma performance otimizada para operações frequentes de leitura e escrita, tornando-o ideal para aplicações que requerem um alto desempenho.
- Armazenamento local: os bancos de dados internos permitem que os dados sejam acessados rapidamente sem a necessidade de conexão com a internet. Isso os torna ideais para o armazenamento de dados críticos e que necessitam de alta performance. Aplicativos que exigem acesso rápido a grandes volumes de dados, como jogos e aplicações de mídia, se beneficiam enormemente desse tipo de armazenamento.

Firestore

É um banco de dados NoSQL baseado em nuvem que oferece sincronização em tempo real e suporte offline, sendo ideal para aplicativos móveis. Ele armazena dados em documentos, que são organizados em coleções. Cada documento contém pares de chave-valor e pode incluir subcoleções.

O Firestore permite a sincronização em tempo real entre dispositivos, o que facilita a construção de aplicativos colaborativos que precisam de dados sempre atualizados. A estrutura de dados NoSQL do Firestore é altamente flexível, permitindo o armazenamento de dados complexos em documentos JSON.

Além disso, ele é altamente escalável, suportando aplicativos que crescem rapidamente em termos de volume de dados e número de usuários. O Firestore permite a sincronização em tempo real, garantindo que todas as instâncias do aplicativo tenham acesso aos dados mais recentes. Isso é particularmente útil em aplicativos de colaboração e compartilhamento de dados em tempo real.

Ferramentas e tecnologias

Firestore SDK

O Firestore SDK fornece uma API simples e robusta para interagir com o Firestore. Ele suporta operações de CRUD (Create, Read, Update, Delete) e sincronização em tempo real com suporte offline. Isso significa que os dados podem ser acessados e modificados mesmo quando o dispositivo está desconectado da internet, sincronizando automaticamente quando a conexão é restabelecida.

- **Operações CRUD:** suporta criação, leitura, atualização e exclusão de dados de forma eficiente.
- **Sincronização em tempo real:** atualiza os dados em todos os dispositivos conectados instantaneamente.
- **Suporte offline:** permite que os dados sejam acessados e modificados sem conexão à internet, sincronizando posteriormente.

Estrutura de dados NoSQL

Firestore usa uma estrutura de dados NoSQL, onde os dados são armazenados em documentos JSON organizados em coleções. Isso proporciona flexibilidade para armazenar dados complexos e aninhados, e permite consultas eficientes.

- **Documentos e coleções:** dados são organizados em documentos, que são agrupados em coleções.
- **Pares de chave-valor:** cada documento contém pares de chave-valor, permitindo uma estrutura de dados flexível.
- **Subcoleções:** documentos podem conter subcoleções, permitindo a organização hierárquica dos dados.

Configuração do Firestore

Teoria e configuração - Criar um projeto no Firebase

Para usar o Firestore, é necessário criar um projeto no Firebase Console. Isso envolve registrar o aplicativo e configurar as credenciais de autenticação.

1. **Configurar Firestore no Firebase Console:** no Firebase Console, navegue até a seção Firestore Database e clique em "Criar banco de dados". Escolha o modo de segurança (produção ou teste) e configure as regras de segurança.
2. **Adicionar Firestore ao projeto:** adicione o Firebase SDK ao seu projeto móvel. Isso inclui configurar o arquivo de configuração do Firebase e inicializar o Firebase no aplicativo.
3. **Exemplos de Configuração Firebase Console**
 - Passo 1: crie um projeto no Firebase Console.
 - Passo 2: navegue até a seção "Firestore Database" e clique em "Criar Banco de Dados".
 - Passo 3: escolha o modo de segurança e configure as regras de segurança.
4. **Adicionar Firebase ao projeto**
 - Passo 1: adicione o Firebase SDK ao projeto. Para aplicativos Android, adicione o arquivo google-services.json ao diretório app e modifique o arquivo build.gradle conforme as instruções fornecidas pelo Firebase.
 - Passo 2: Inicialize o Firebase no aplicativo:

```
import firebase from 'firebase/app';
import 'firebase/firestore';

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_AUTH_DOMAIN",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_STORAGE_BUCKET",
  messagingSenderId: "YOUR_MESSAGING_SENDER_ID",
```



```
appId: "YOUR_APP_ID"
};

firebase.initializeApp(firebaseConfig);
const db = firebase.firestore();
```

Operações Básicas com Firestore - Teoria e implementação

Criação de documentos

- Processo: adicionar um novo documento a uma coleção no Firestore. Cada documento é identificado por um ID exclusivo.

Leitura de documentos

- Processo: recuperar documentos de uma coleção. Isso pode ser feito para um único documento ou para todos os documentos em uma coleção.

Atualização de documentos

- Processo: atualizar os campos de um documento existente no Firestore. As atualizações podem ser feitas em um ou mais campos.

Exclusão de documentos

- Processo: remover um documento de uma coleção.

Atenção: a exclusão é permanente e não pode ser desfeita.

Exemplos de implementação

Criação de documentos:

```
db.collection("users").add({
  first: "Ada",
  last: "Lovelace",
  born: 1815
```

```
    })  
    .then((docRef) => {  
      console.log("Document written with ID: ", docRef.id);  
    })  
    .catch((error) => {  
      console.error("Error adding document: ", error);  
    });  
  });
```

Leitura de documentos:

```
db.collection("users").get().then((querySnapshot) => {  
  querySnapshot.forEach((doc) => {  
    console.log(`${doc.id} => ${doc.data()}`);  
  });  
});
```

Atualização de documentos:

```
var userRef = db.collection("users").doc("ABC123");  
  
// Atualiza um campo  
userRef.update({  
  last: "Byron"  
})  
  
.then(() => {  
  console.log("Document successfully updated!");  
})  
  
.catch((error) => {  
  console.error("Error updating document: ", error);  
});
```

Exclusão de documentos:

```
db.collection("users").doc("ABC123").delete().then(() => {  
  console.log("Document successfully deleted!");  
});
```

```
}).catch((error) => {  
  console.error("Error removing document: ", error);  
});
```

EMSE

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, abordamos a conexão de aplicativos móveis com bancos de dados internos e o Firebase Firestore. Exploramos a importância dos bancos de dados, a diferença entre bancos de dados locais e Firestore, e como configurar e usar Firestore em um aplicativo móvel. Discutimos operações básicas com Firestore e as melhores práticas para gerenciamento de dados de forma eficiente e segura.



REFERÊNCIAS

DEMETRESCU, D. **Firestore Cookbook: Over 70 recipes to help you create real-time web and mobile applications with Firestore**. [s.l.]: Packt Publishing, 2017.

MURPHY, M. **The Busy Coder's Guide to Android Development**. [s.l.]: CommonsWare, LLC, 2019.

SHAH, A. **Firestore Essentials - Android Edition**. [s.l.]: Leanpub, 2015.

EMANIP

PALAVRAS-CHAVE

React native. Front-end. Firestore.

EXEMPLO

POS TECH